

Fondamenti di Informatica T2

Lab08 – EDLift

Corso di Laurea in Ingegneria Informatica

Anno accademico 2023/2024

Prof. ROBERTA CALEGARI

Prof. AMBRA MOLESINI

Dipartimento di Informatica – Scienza e Ingegneria (DISI)



Obiettivo dell'esercitazione

- **OBIETTIVO: realizzare una applicazione che simuli le politiche di funzionamento di diversi tipi di ascensori**
 - Vari tipi di ascensori:
 - BASIC
 - SALUTISTA
 - MULTIPIANO
 - [in futuro, magari, MULTIPIANO SMART]
 - ...

Analisi del dominio

- Tutti sappiamo cosa fa un ascensore (forse... 😊)
 - Lo si chiama a un certo *piano*
 - Se è possibile, la *chiamata* viene *accettata*
 - L'ascensore *inizia a muoversi* verso il piano richiesto
 - Dopo un certo tempo, l'ascensore *raggiunge* il piano richiesto
 - L'ascensore *si ferma* per far scendere i passeggeri
- Entro tale schema sono possibili varie ***politiche di funzionamento***
 - Mentre l'ascensore si sta muovendo, *accetta altre chiamate?*
 - L'ascensore *accetta una sola chiamata per volta* o è in grado anche di *gestire più chiamate contemporaneamente?* (e se sì, con quale politica?)
 - Altre varianti...?

Analisi del dominio

- Ogni ascensore è installato in un ben preciso *edificio*
 - Ogni edificio ha un numero ben preciso di *piani*
 - I piani seguono una ben precisa *numerazione*, che varia da un edificio all'altro (anche a parità di numero totale di piani)
 - Una palazzina di 4 piani fuori terra avrebbe i piani numerati da ... a ...?
 - Una palazzina di 4 piani ma col seminterrato per cantina e garage...?
- Ogni edificio ha dunque il suo *schema di numerazione dei piani*
 - L'ascensore si può muovere sempre e solo fra il *piano minimo* e il *piano massimo*, comunque siano numerati
 - Esempio: palazzina di 4 piani numerati da 0 a 3
 - Esempio: palazzina di 4 piani con seminterrato, piani numerati da -1 a 2
 - Il numero totale di piani è ***PianoMax-PianoMin+1***



L'applicazione EDLift

- **L'applicazione è un simulatore di ascensori**
 - La parte grafica è data già fatta nello start kit (in futuro la farete voi)
 - Il motore di simulazione è dato già fatto nel controller
- **OBIETTIVO: realizzare le politiche di funzionamento di diversi tipi di ascensori**
 - Vari tipi di ascensori: BASIC, SALUTISTA, MULTIPIANO
 - (in futuro magari anche altri, es. MULTIPIANO SMART...)
 - Ogni tipo specifico di ascensore incorpora la logica con cui quel determinato tipo di ascensore accetta (se l'accetta) la/le chiamata/e a quel piano e la/le serve



Tipi di ascensori (1/2)

- **L'ascensore BASIC**

- È quello installato nella maggior parte delle case e dei condomini
- Accetta una sola chiamata per volta e la serve
- Mentre sta servendo una chiamata, non ne accetta altre

- **L'ascensore SALUTISTA (un'esclusiva di EDLift!!) 😊**

- *Promuove uno stile di vita sano, costringendoti a fare sempre almeno un piano a piedi 😊*
- Di conseguenza, se la richiesta è di salire al 4° piano, ti lascia al 3°; se è di scendere dal 4° al 1°, ti lascia al 2°; e così via

(segue)

Tipi di ascensori (2/2)

- **L'ascensore MULTIPIANO**

- Accetta anche più chiamate per volta, e le serve *in ordine*
- Rispetto al BASIC, accetta prenotazioni per chiamate anche mentre è in movimento per servirne altre precedenti
- Però, non lo fa in modo furbo: poiché serve le chiamate nell'ordine in cui gli arrivano, può fare molti su/giù inutili (es. piano 2,8,4,9, etc.)

- **L'ascensore MULTIPIANO SMART**

- **Non implementato in questa esercitazione** (magari in futuro...)
- È quello installato solitamente negli hotel e nei grossi palazzi
- Accetta anche più chiamate per volta, ma le serve *in modo intelligente*
- Non segue l'ordine con cui gli sono arrivate: tiene conto della direzione in cui si sta muovendo e dei piani che toccherebbe comunque (es. se si sta muovendo dal piano 4 al piano 9, può servire richieste per i piani intermedi 5, 7, 8, ma non per il piano 2, che servirà dopo; etc.)

Ascensori & ascensori

Ma come modello tanti
tipi di ascensori diversi?

Non sarebbe male poter
usare l'ereditarietà..





La gerarchia di ascensori

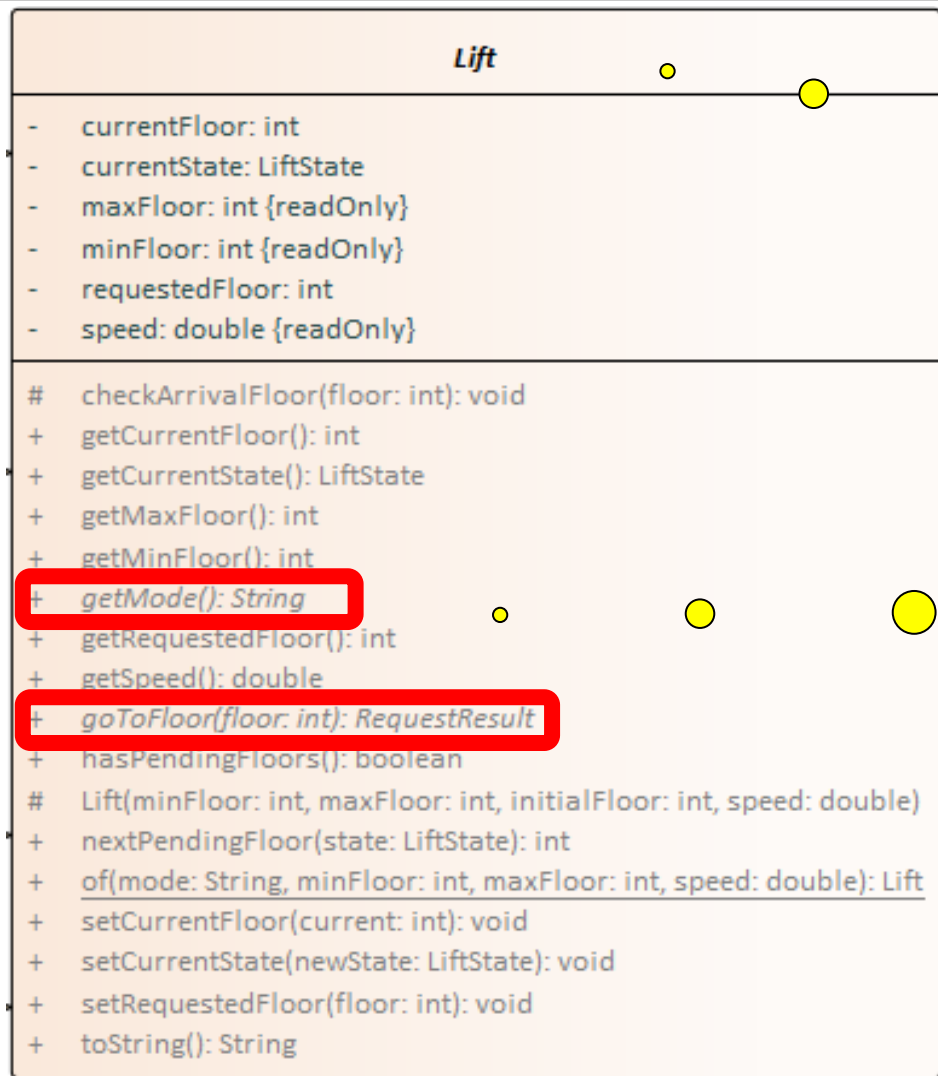
- La classe base della gerarchia è `Lift`
- Vari tipi di ascensori: BASIC, SALUTISTA, MULTIPIANO
- Ogni ascensore specifico specializza la classe base `Lift`
 - Metodo `goToFloor`: incorpora la logica con cui l'ascensore accetta (se l'accetta) la chiamata a quel piano e la serve



La classe base (astratta) Lift

- Rappresenta un ascensore *prescindendo dalla sua politica di funzionamento*, espressa dal metodo (astratto) `goToFloor`
- Proprietà «esterne», fornite all'atto della costruzione:
 - Numero del piano minimo dell'edificio in cui è installato
 - Numero del piano massimo dell'edificio in cui è installato
 - Numero del piano iniziale (a cui si trova all'inizio della simulazione)
 - La velocità a cui si muove (in m/s) ← **non usata in questa esercitazione**
- Internamente, nel suo stato, esso mantiene inoltre:
 - Lo stato attuale (fermo, in salita, in discesa)
 - Il piano attuale (compreso ovviamente fra minimo e massimo, inclusi)
 - Il piano richiesto (a cui gli è stato chiesto di andare)

La classe base Lift



Classe
Astratta

Metodi
Astratti



La classe base (astratta) Lift

- Metodi di utilità
 - toString descrive l'ascensore in questione
- Metodi per gestire eventuali richieste multiple
 - hasPendingFloors / nextPendingFloor
 - Utili per quegli ascensori che accettano più richieste
 - Implementazioni «fake» in Lift, utili per quegli ascensori che servono una sola chiamata e quindi non hanno mai richieste pendenti
- Soprattutto, la classe internalizza una factory
 - Il metodo **of** restituisce un'istanza del «giusto» (sotto-)tipo di ascensore in base agli argomenti specificati
 - NB: per catturare eventuali casi non previsti si fa uso di un meccanismo (le eccezioni) che studieremo più avanti

La classe base (astratta) Lift

- Metodi di utilità
 - toString descrive l'ascensore in questione
- Metodi per gestire eventuali richieste multiple
 - hasPendingFloors / nextPendingFloor
 - Utili per quegli ascensori che accettano più richieste
 - Implementazioni «fake» in Lift, utili per quegli ascensori che servono una sola chiamata e quindi non hanno mai richieste pendenti
- Soprattutto, la classe internalizza una factory

```
public static Lift of(String mode, int minFloor, int maxFloor, double speed) {  
    int initialFloor = (minFloor<=0 && maxFloor>=1) ? 0 : minFloor;  
    return switch(mode) {  
        case "BASIC"      -> new BasicLift(minFloor, maxFloor, initialFloor, speed);  
        case "HEALTHY"    -> new HealthyLift(minFloor, maxFloor, initialFloor, speed);  
        case "MULTIFLOOR" -> new MultiFloorLift(minFloor, maxFloor, initialFloor, speed);  
        // NB: il caso default sfrutta anche qui il meccanismo delle eccezioni (che vedremo più avanti)  
        // per lanciare un allarme relativo alla presenza di un argomento illegale  
        default           -> throw new IllegalArgumentException("Unsupported lift mode");  
    };  
}
```

Gli ascensori concreti

- Gli ascensori concreti da realizzare sono tre:
BASIC, HEALTHY, MULTIFLOOR (in futuro, magari, anche SMART)
 - L'ascensore **BASIC** accetta una sola chiamata per volta e la serve (mentre sta servendo una chiamata, non ne accetta altre)
 - L'ascensore **HEALTHY** lascia l'utente sempre «un piano prima» di quanto richiesto (se la richiesta è di salire al 4° piano, ti lascia al 3°; se è di scendere dal 4° al 1°, ti lascia al 2°; e così via)
 - L'ascensore **MULTIFLOOR** accetta anche più chiamate e le serve nell'ordine in cui le ha ricevute
 - necessità di appoggiarsi a una struttura a **coda** (politica FIFO: First In, First Out)
Come realizzarla...?

BasicLift

BasicLift

```
+ BasicLift(minFloor: int, maxFloor: int, initialFloor: int, speed: double)
+ getMode(): String
+ goToFloor(floor: int): RequestResult
```

- Ridefinisce opportunamente il metodo **goToFloor**:
 - accetta una chiamata solo se è fermo: altrimenti la rifiuta
 - se l'accetta, imposta il piano a cui andare con **setRequestedFloor**
- Ridefinisce opportunamente il metodo **getMode**:
 - restituisce la stringa «Basic»

BasicLift

- La classe **BasicLift** concretizza **Lift** nel caso specifico dell'ascensore basico:
 - il costruttore accetta gli stessi argomenti del costruttore di Lift
 - la logica di servizio delle chiamate espressa da **goToFloor** deve:
 - se l'ascensore è fermo, impostare il piano di destinazione desiderato e restituire ACCEPTED
 - altrimenti, restituire REJECTED

HealthyLift

HealthyLift

```
+ getMode(): String  
+ goToFloor(floor: int): RequestResult  
+ HealthyLift(minFloor: int, maxFloor: int, initialFloor: int, speed: double)
```

- **Ridefinisce opportunamente il metodo goToFloor:**
 - rifiuta una chiamata non solo se è già in movimento, ma anche se la destinazione dista dal piano attuale meno di un piano.
 - se l'accetta, imposta il piano a cui andare al «piano prima» di quello effettivamente richiesto dall'utente
(NB: qual è il «piano prima»...?)
- **Ridefinisce opportunamente il metodo getMode:**
 - restituisce la stringa «Healthy»

HealthyLift

- La classe ***HealthyLift*** concretizza ***Lift*** nel caso specifico dell'ascensore salutista
- Il funzionamento è analogo a quello dell'ascensore base, inclusa la scelta di non gestire chiamate pendenti
- L'unica differenza è nella logica di funzionamento incapsulata nel metodo **goToFloor**

HealthyLift

- Il metodo `goToFloor` deve:
 - se l'ascensore non è fermo, rifiutare la chiamata restituendo `REJECTED`
 - se, invece, l'ascensore è fermo:
 - calcolare la differenza in piani fra il piano richiesto e quello corrente
 - se essa è non superiore a 1, rifiutare la chiamata
 - se è superiore, calcolare il nuovo piano di arrivo tenendo conto del requisito di far fare sempre un piano a piedi, impostarlo con `setRequestedFloor` e restituire il risultato `MODIFIED` integrando in esso un apposito messaggio e il valore del piano di destinazione calcolato (metodi `setMsg/setFloor`)

MultiFloorLift

MultiFloorLift	
-	queue: Queue
+	getMode(): String
+	goToFloor(floor: int): RequestResult
+	hasPendingFloors(): boolean
+	MultiFloorLift(minFloor: int, maxFloor: int, initialFloor: int, speed: double)
+	nextPendingFloor(state: LiftState): int

- **MultiFloorLift** concretizza **Lift** nel caso specifico dell'ascensore che accetta più chiamate e le serve nell'ordine in cui gli sono pervenute
- Funzionamento analogo a quello dell'ascensore base, **ma** **abbisogna di una coda** per registrare ed estrarre le chiamate pendenti
- Anche qui, la logica di funzionamento è incapsulata nel metodo **goToFloor**



MultiFloorLift

- Il metodo `goToFloor` deve:
 - se l'ascensore non è fermo
 - tenta di inserire la nuova prenotazione in coda
 - se ci riesce, restituire `ACCEPTED`
 - se non ci riesce (coda piena), restituire `REJECTED` con apposito messaggio d'errore (e piano convenzionalmente pari a `Integer.MIN_VALUE`)
 - se l'ascensore è fermo: impostare il piano di destinazione desiderato e restituire `ACCEPTED` (come nell'ascensore `BASIC`)

N.B: maggiori dettagli riguardo l'algoritmo saranno forniti nella seconda parte dell'esercitazione

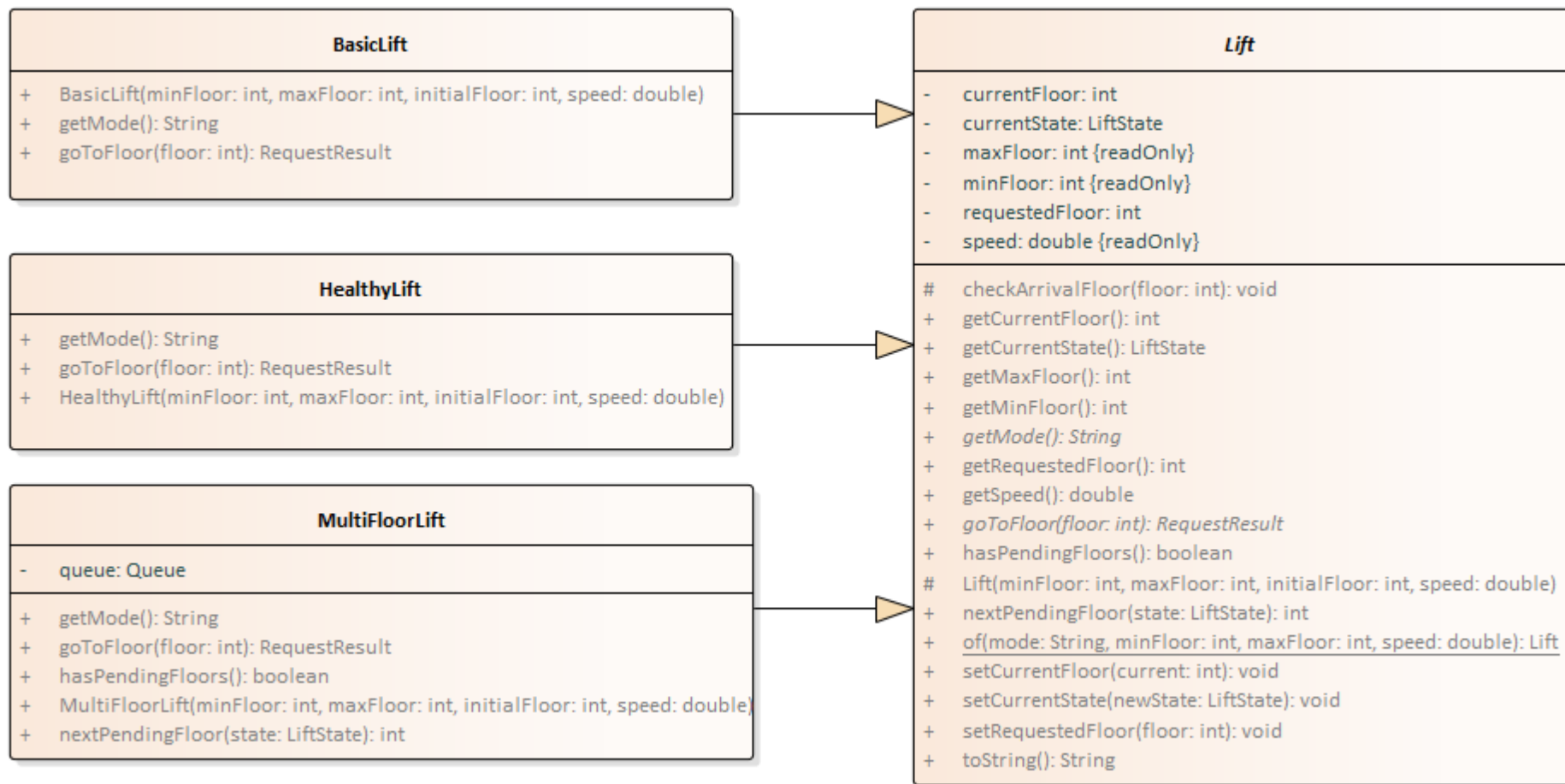


Separazione dei compiti

Chi fa cosa

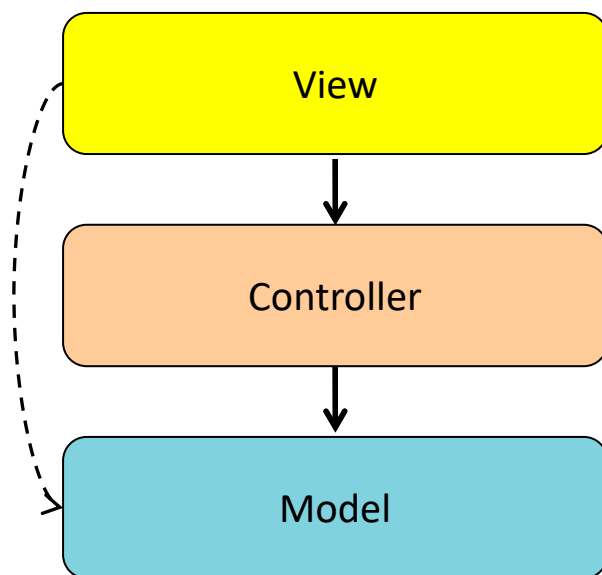
- Gli ascensori:
 - Stabiliscono la logica con cui si va al prossimo piano (`goToFloor`)
 - Stabiliscono, se utile, la logica con cui accettare più richieste (`hasPendingFloors` / `nextPendingFloor`) appoggiandosi, se necessario, a proprie strutture dati (`code` , `pool` etc.)
 - NB: la classe base `Lift` fornisce già implementazioni «fake» per quegli ascensori che non ne hanno bisogno
- Il controller
 - Gestisce la simulazione simulando lo scorrere del tempo e l'avanzamento dell'ascensore
 - Chiede all'ascensore, se e quando necessario, il «prossimo» piano da servire (`hasPendingFloors` / `nextPendingFloor`)

La gerarchia di classi



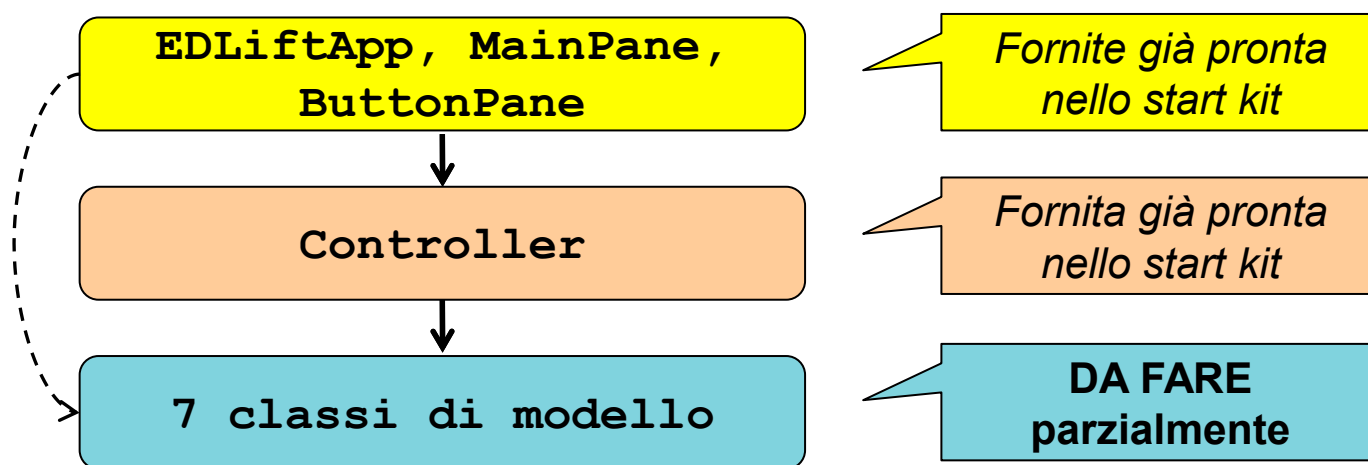
La struttura dell'applicazione

- L'architettura software non è banale, ci sono parecchie entità da modellare e responsabilità diverse da attribuire
- Occorre utilizzare nuovamente il Pattern MVC per organizzare al meglio la nostra applicazione e suddividere nel modo corretto le responsabilità

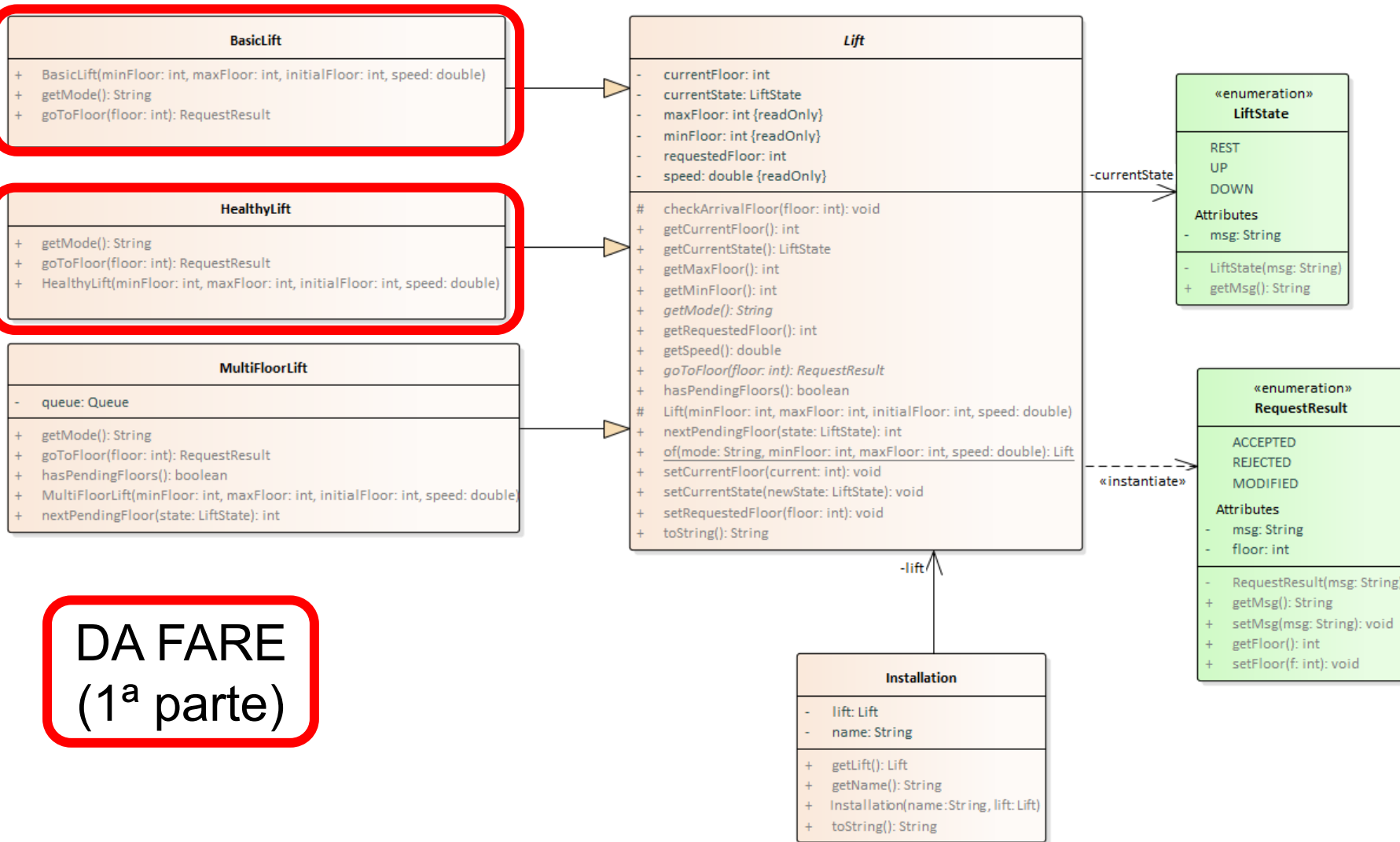


EDLift con MVC

- Nel caso della nostra applicazione EDLift:
 - **Model:** svariate classi che memorizzano lo stato dell'ascensore
 - **View:** package `ui` realizzato con JavaFX (*fornito nello start kit*)
 - **Controller:** classe **Controller** (*fornito nello start kit*)



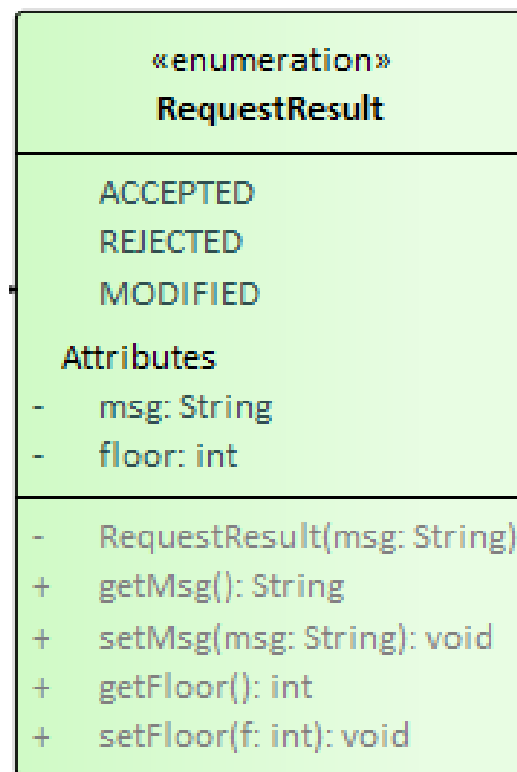
Architettura del model



DA FARE
(1^a parte)

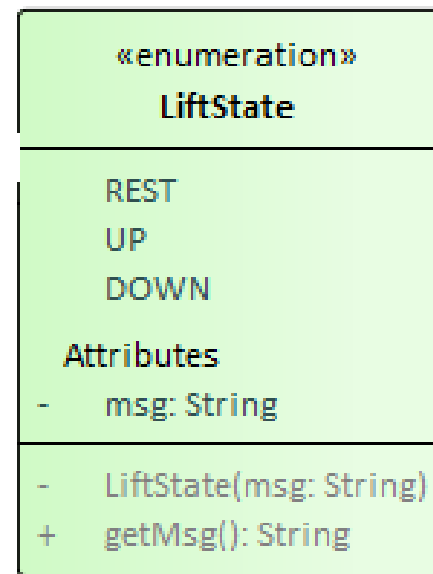
RequestResult

- ***RequestResult*** definisce i tre risultati possibili: ***ACCEPTED***, ***REJECTED*** e ***MODIFIED***
- E' possibile impostare sia un messaggio (utile soprattutto per ***MODIFIED***), sia il valore del piano a cui andrà l'ascensore



LiftState

- **LiftState** definisce i tre stati possibili
 - **REST:** fermo al piano
 - **UP:** in salita
 - **DOWN:** in discesa
- Anche qui è previsto un messaggio incapsulato, accessibile tramite apposito accessor getMsg



Installation

- ***Installation*** è una pura classe-dati che incapsula i dati della specifica installazione dell'ascensore in quell'edificio
- Disponibili gli appositi accessor e toString

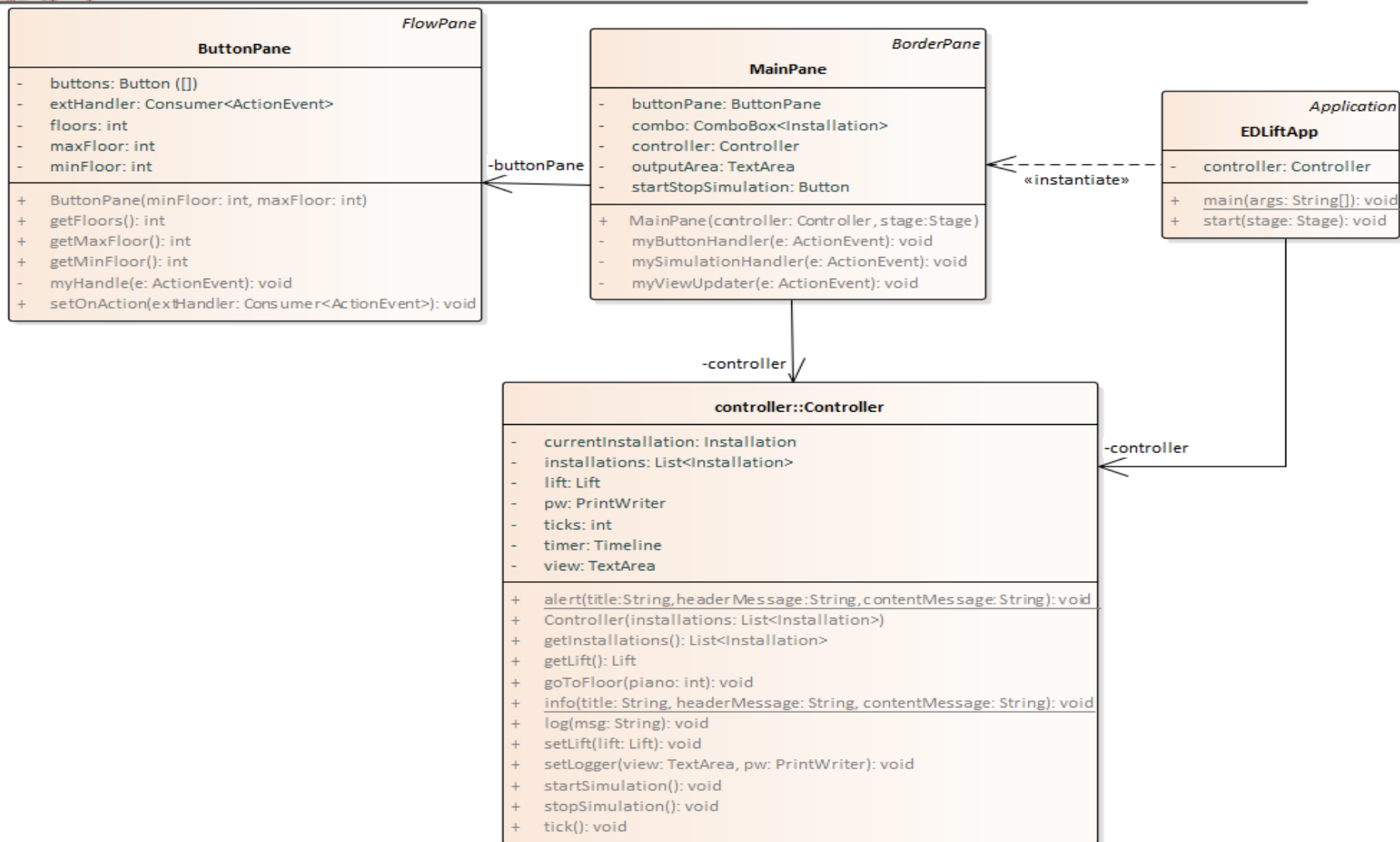
Installation	
-	lift: Lift
-	name: String
+	getLift(): Lift
+	getName(): String
+	Installation(name:String, lift: Lift)
+	toString(): String

Il Controller

- Contiene il motore di simulazione
- Svolge il classico compito del «grande burattinaio» tra model e view

Controller
- currentInstallation: Installation - installations: List<Installation> - lift: Lift - pw: PrintWriter - ticks: int - timer: Timeline - view: TextArea
+ <u>alert(title: String, headerMessage: String, contentMessage: String): void</u> + Controller(installations: List<Installation>) + getInstallations(): List<Installation> + getLift(): Lift + goToFloor(piano: int): void + <u>info(title: String, headerMessage: String, contentMessage: String): void</u> + log(msg: String): void + setLift(lift: Lift): void + setLogger(view: TextArea, pw: PrintWriter): void + startSimulation(): void + stopSimulation(): void + tick(): void

La view



Esempi di funzionamento

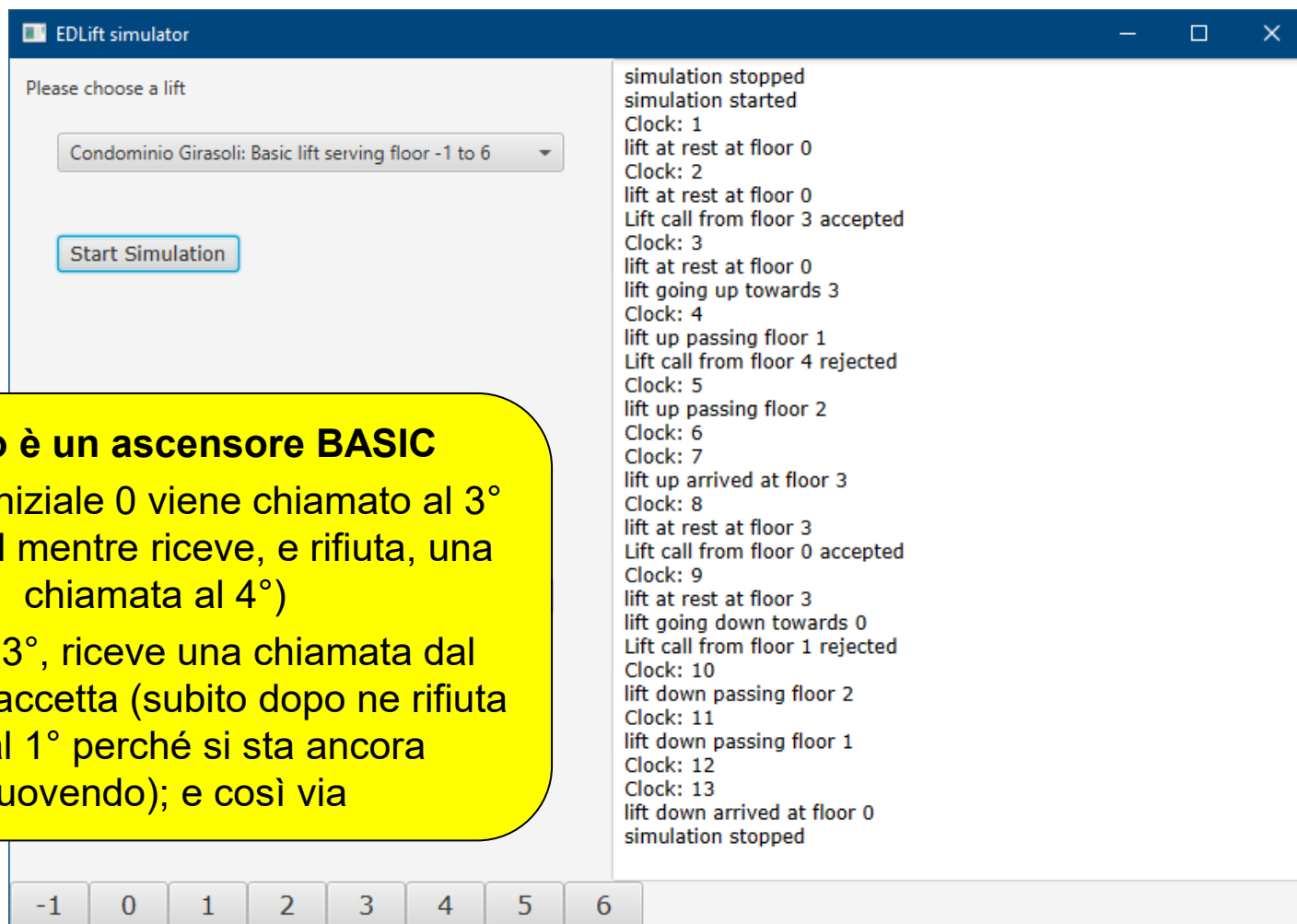
The image displays three screenshots of the EDLift simulator interface, illustrating different lift configurations and floor ranges. Each screenshot shows a dropdown menu for selecting a lift and a 'Start Simulation' button. The floor ranges are indicated by callout boxes and floor buttons at the bottom of the interface.

Left Screenshot: The dropdown menu shows 'Hotel Miralago: Healthy lift serving floor -2 to 4'. A callout box points to the dropdown menu with the text: 'La pulsantiera è configurata in base alla specifica installazione'. The floor buttons at the bottom range from -2 to 4.

Top Right Screenshot: The dropdown menu shows 'Condominio Girasoli: Basic lift serving floor -1 to 6'. A callout box points to the dropdown menu with the text: 'Qui i piani vanno da -1 a 6'. The floor buttons at the bottom range from -1 to 2.

Bottom Right Screenshot: The dropdown menu shows 'Palazzina Ferrari: Basic lift serving floor 0 to 4'. A callout box points to the dropdown menu with the text: 'Qui i piani vanno da 0 a 4'. The floor buttons at the bottom range from 0 to 4.

Esempi di funzionamento



The screenshot shows the EDLift simulator window. On the left, there is a control panel with a dropdown menu set to 'Condominio Girasoli: Basic lift serving floor -1 to 6' and a 'Start Simulation' button. On the right, a log displays the simulation's progress, including clock times and lift movements. At the bottom, a floor indicator shows buttons for floors -1, 0, 1, 2, 3, 4, 5, and 6.

EDLift simulator

Please choose a lift

Condominio Girasoli: Basic lift serving floor -1 to 6

Start Simulation

simulation stopped
simulation started
Clock: 1
lift at rest at floor 0
Clock: 2
lift at rest at floor 0
Lift call from floor 3 accepted
Clock: 3
lift at rest at floor 0
lift going up towards 3
Clock: 4
lift up passing floor 1
Lift call from floor 4 rejected
Clock: 5
lift up passing floor 2
Clock: 6
Clock: 7
lift up arrived at floor 3
Clock: 8
lift at rest at floor 3
Lift call from floor 0 accepted
Clock: 9
lift at rest at floor 3
lift going down towards 0
Lift call from floor 1 rejected
Clock: 10
lift down passing floor 2
Clock: 11
lift down passing floor 1
Clock: 12
Clock: 13
lift down arrived at floor 0
simulation stopped

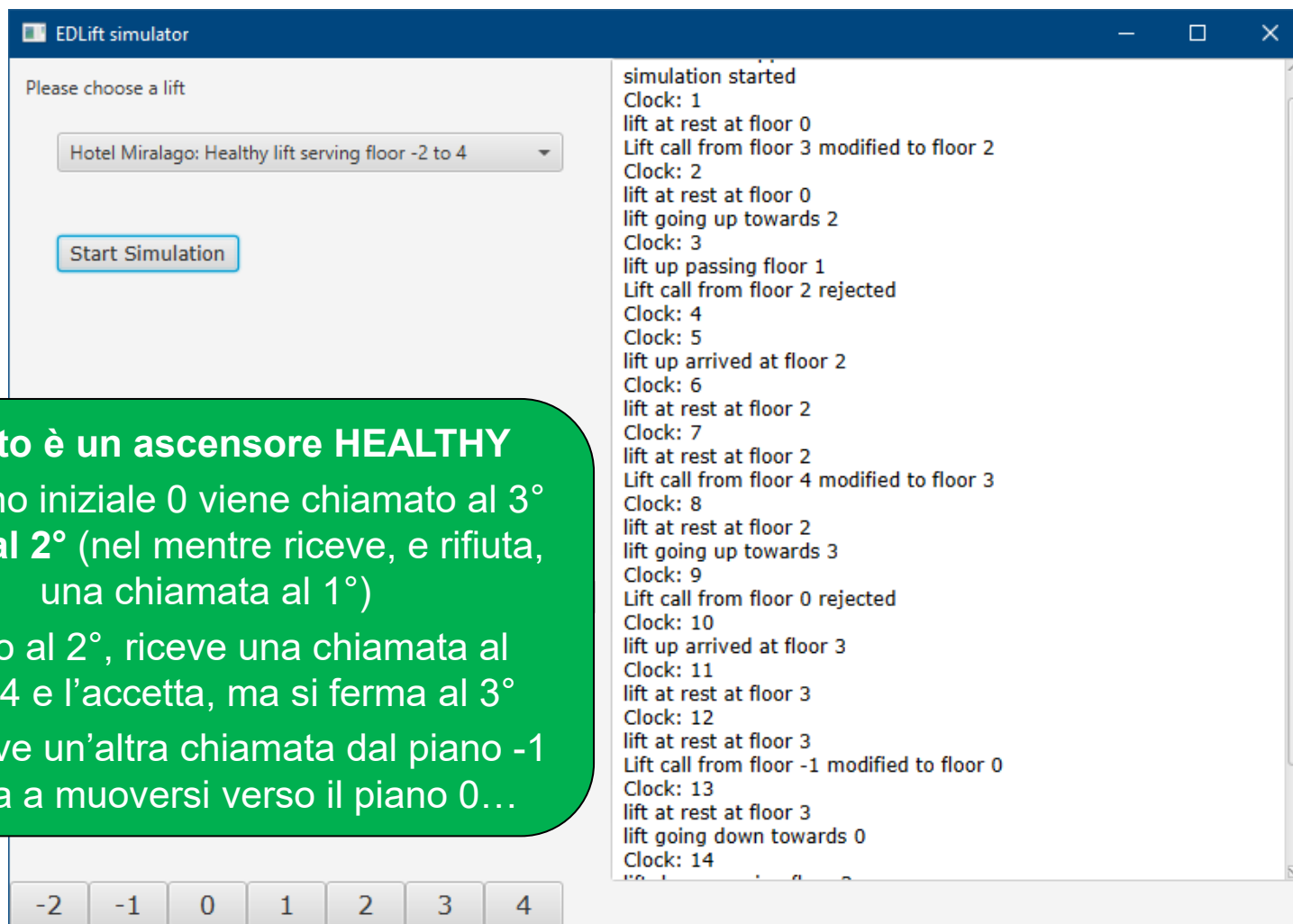
-1 0 1 2 3 4 5 6

Questo è un ascensore BASIC

Dal piano iniziale 0 viene chiamato al 3° e ci va (nel mentre riceve, e rifiuta, una chiamata al 4°)

Giunto al 3°, riceve una chiamata dal piano 0 e l'accetta (subito dopo ne rifiuta una dal 1° perché si sta ancora muovendo); e così via

Esempi di funzionamento



The screenshot shows the EDLift simulator window. On the left, there is a control panel with a dropdown menu set to "Hotel Miralago: Healthy lift serving floor -2 to 4" and a "Start Simulation" button. Below this is a row of buttons for floors -2, -1, 0, 1, 2, 3, and 4. On the right, a log displays the simulation's progress, including clock times and lift actions.

simulation started
Clock: 1
lift at rest at floor 0
Lift call from floor 3 modified to floor 2
Clock: 2
lift at rest at floor 0
lift going up towards 2
Clock: 3
lift up passing floor 1
Lift call from floor 2 rejected
Clock: 4
Clock: 5
lift up arrived at floor 2
Clock: 6
lift at rest at floor 2
Clock: 7
lift at rest at floor 2
Lift call from floor 4 modified to floor 3
Clock: 8
lift at rest at floor 2
lift going up towards 3
Clock: 9
Lift call from floor 0 rejected
Clock: 10
lift up arrived at floor 3
Clock: 11
lift at rest at floor 3
Clock: 12
lift at rest at floor 3
Lift call from floor -1 modified to floor 0
Clock: 13
lift at rest at floor 3
lift going down towards 0
Clock: 14

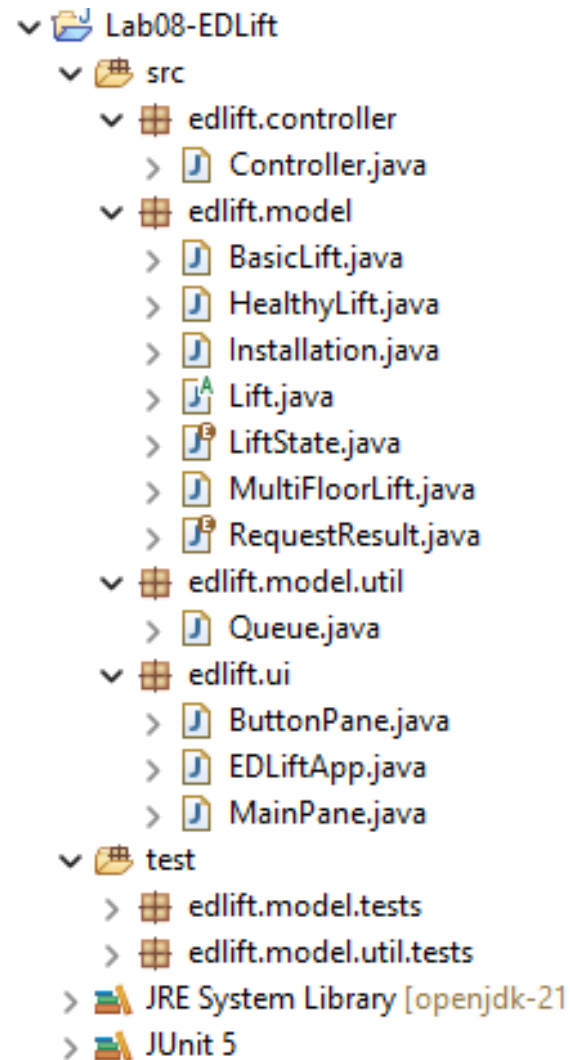
Questo è un ascensore **HEALTHY**

Dal piano iniziale 0 viene chiamato al 3°
ma va al 2° (nel mentre riceve, e rifiuta,
una chiamata al 1°)

Giunto al 2°, riceve una chiamata al
piano 4 e l'accetta, ma si ferma al 3°
Poi riceve un'altra chiamata dal piano -1
e inizia a muoversi verso il piano 0...

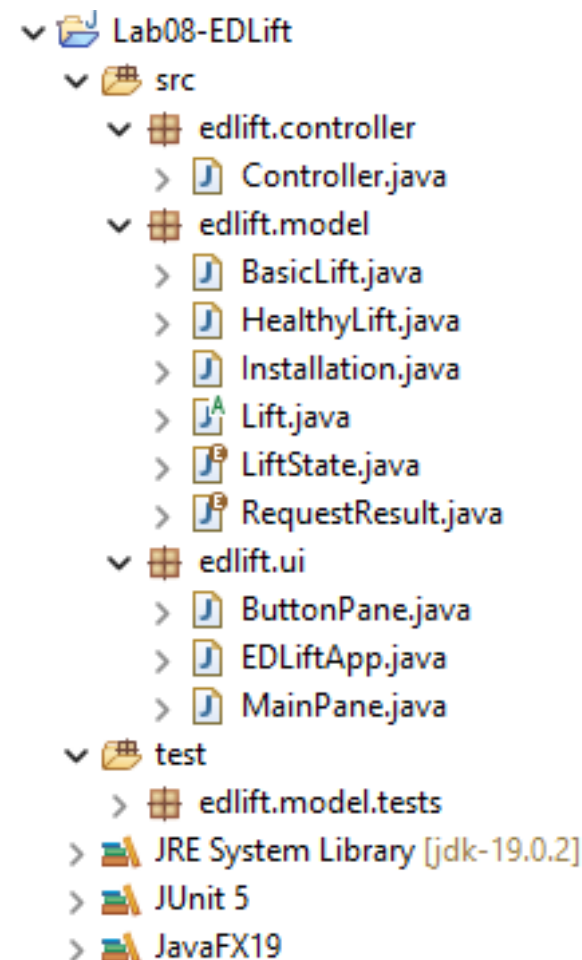
Lo startkit (1ª parte)

- Classica organizzazione dell'ambiente di lavoro con due source folder per sorgenti e test.
- Organizzazione in package:
 - Package model (da completare)
 - Package controller già pronto nello startkit
 - Package ui (la view) fornito nello start kit



Lo startkit (1ª parte)

- **Le classi da fare (per ora)**
 - **BasicLift** (semplice)
 - **HealthyLift** (decisamente più complicata)
 - ...
- Tutte devono implementare:
 - **getMode**, che restituisce la stringa relativa al modo di funzionamento («BASIC», «HEALTHY», «MULTI») usata anche nella factory di Lift
 - **goToFloor**
- **Test, test, test!**
 - Ci sono un sacco di test già pronti
 - Deve funzionare TUTTO!



Da Fare

BasicLift

- + BasicLift(minFloor: int, maxFloor: int, initialFloor: int, speed: double)
- + getMode(): String
- + goToFloor(floor: int): RequestResult

HealthyLift

- + getMode(): String
- + goToFloor(floor: int): RequestResult
- + HealthyLift(minFloor: int, maxFloor: int, initialFloor: int, speed: double)

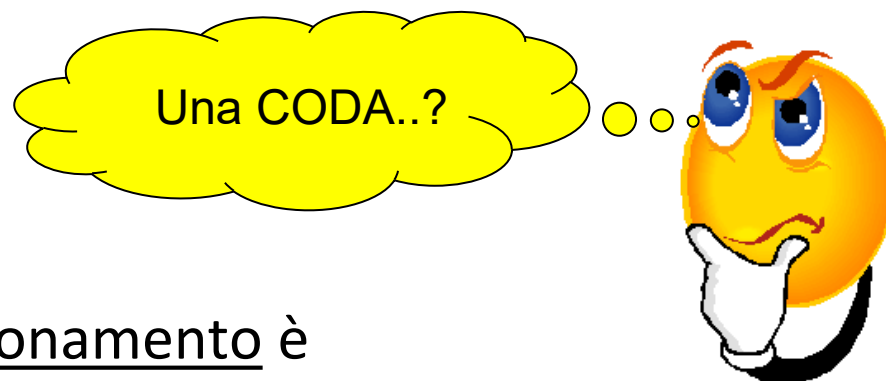
Tempo a disposizione: 50 minuti

Seconda parte

L'ascensore multipiano

MultiFloorLift

- La classe **MultiFloorLift** concretizza **Lift** nel caso specifico dell'ascensore che accetta anche più chiamate e le serve nell'ordine in cui gli sono pervenute
- Il funzionamento è analogo a quello dell'ascensore base, **ma** **abbisogna di una coda** per registrare ed estrarre le chiamate pendenti



- Anche qui, la logica di funzionamento è incapsulata nel metodo goToFloor

MultiFloorLift

- Il metodo `goToFloor` deve:
 - se l'ascensore non è fermo, a differenza dell'ascensore BASIC
 - tentare di inserire la nuova prenotazione in coda
 - se ci riesce, restituire `ACCEPTED`
se non ci riesce (coda piena), restituire `REJECTED` con apposito messaggio d'errore (e piano convenzionalmente pari a `Integer.MIN_VALUE`)
 - se l'ascensore è fermo: impostare il piano di destinazione desiderato e restituire `ACCEPTED` (come nell'ascensore BASIC)

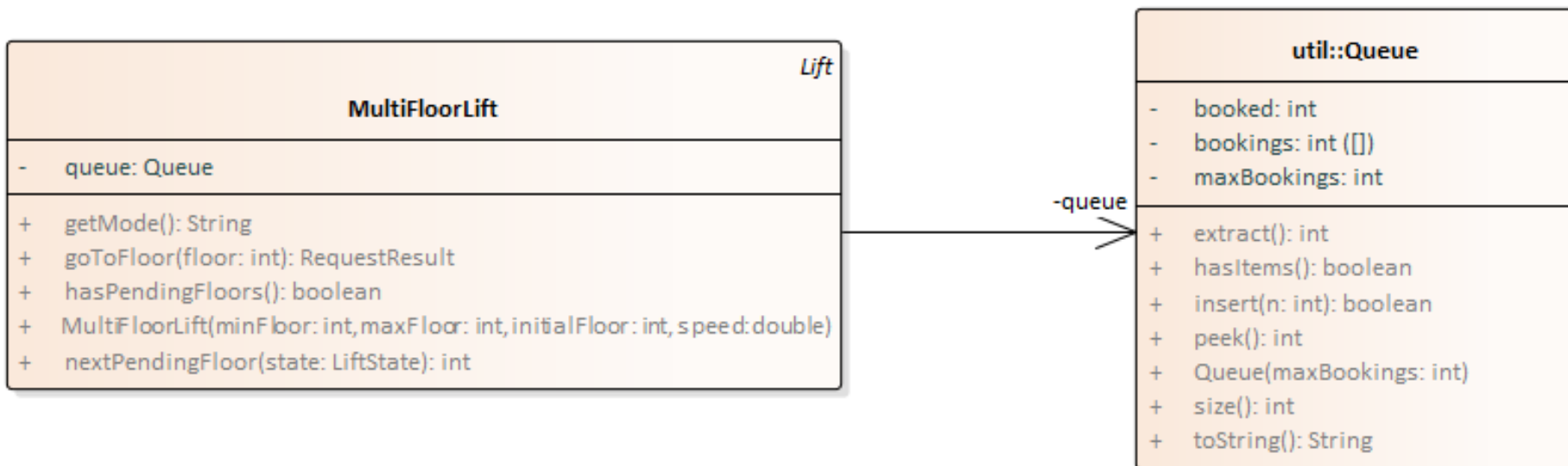
MultiFloorLift

- Il controller, naturalmente, non vede direttamente la coda!
- Esso si rivolge solo al generico ascensore, invocando i due metodi `hasPendingFloors` e `nextPendingFloor`
- Nel caso di MultiFloorLift, `hasPendingFloors` e `nextPendingFloor` rispondono invocando i metodi della coda
- Più precisamente:
 - `hasPendingFloors` verifica se ci sono elementi in coda
 - `nextPendingFloor` estrae il prossimo elemento dalla coda

La «coda»

- Una **coda** è una struttura dati che opera secondo il principio FIFO (First In, First Out)
 - se si inseriscono entità in un certo ordine (es. 2,4,7,3) ...
 - ...saranno estratte nel medesimo ordine (es. 2,4,7,3)
- Per ora non conosciamo strutture dati «collection», quindi dobbiamo *farcela a mano* sfruttando «sotto banco» un array
 - QUALE PROGETTO? → **UML**

MultiFloorLift



La «coda»

– PROGETTO

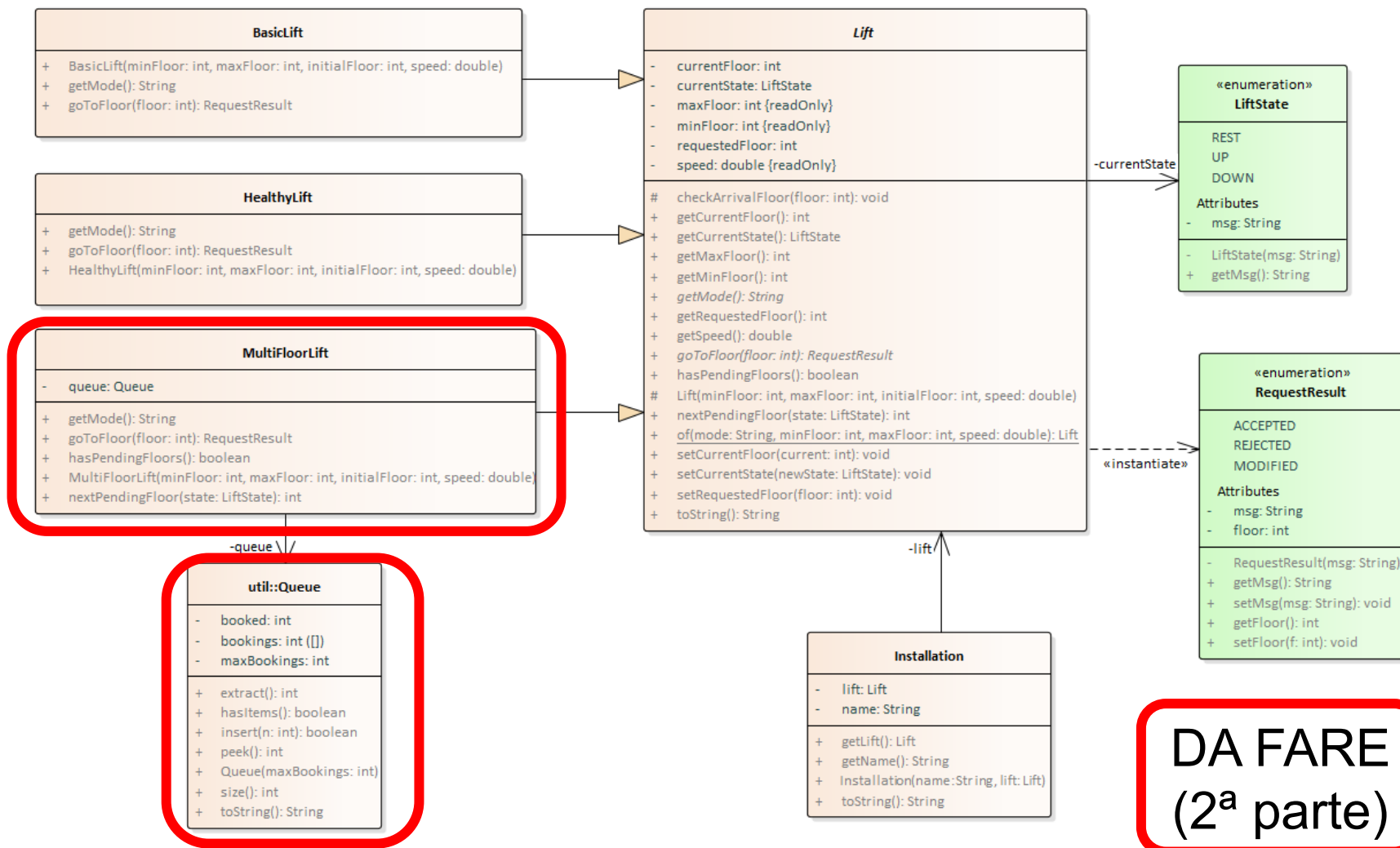
- IPOTESI: la coda ha una capacità massima (non avrebbe senso accettare 100 richieste: l'utente quanto aspetterebbe..?)
- Si inseriscono nuovi elementi «in fondo» all'array → insert
- Si estraggono elementi «dalla testa» dell'array → extract
 - NB: richiede la ricopiatura all'indietro del contenuto residuo!
- Metodi di utilità → size, hasItems, peek
 - size: restituisce il numero di elementi presenti in coda
 - hasItems: restituisce vero se c'è almeno un elemento in coda
 - peek: restituisce il prossimo elemento da estrarre (senza estrarlo)
[non necessaria in questa esercitazione, ma utile per completezza di definizione dell'ADT]

La «coda»

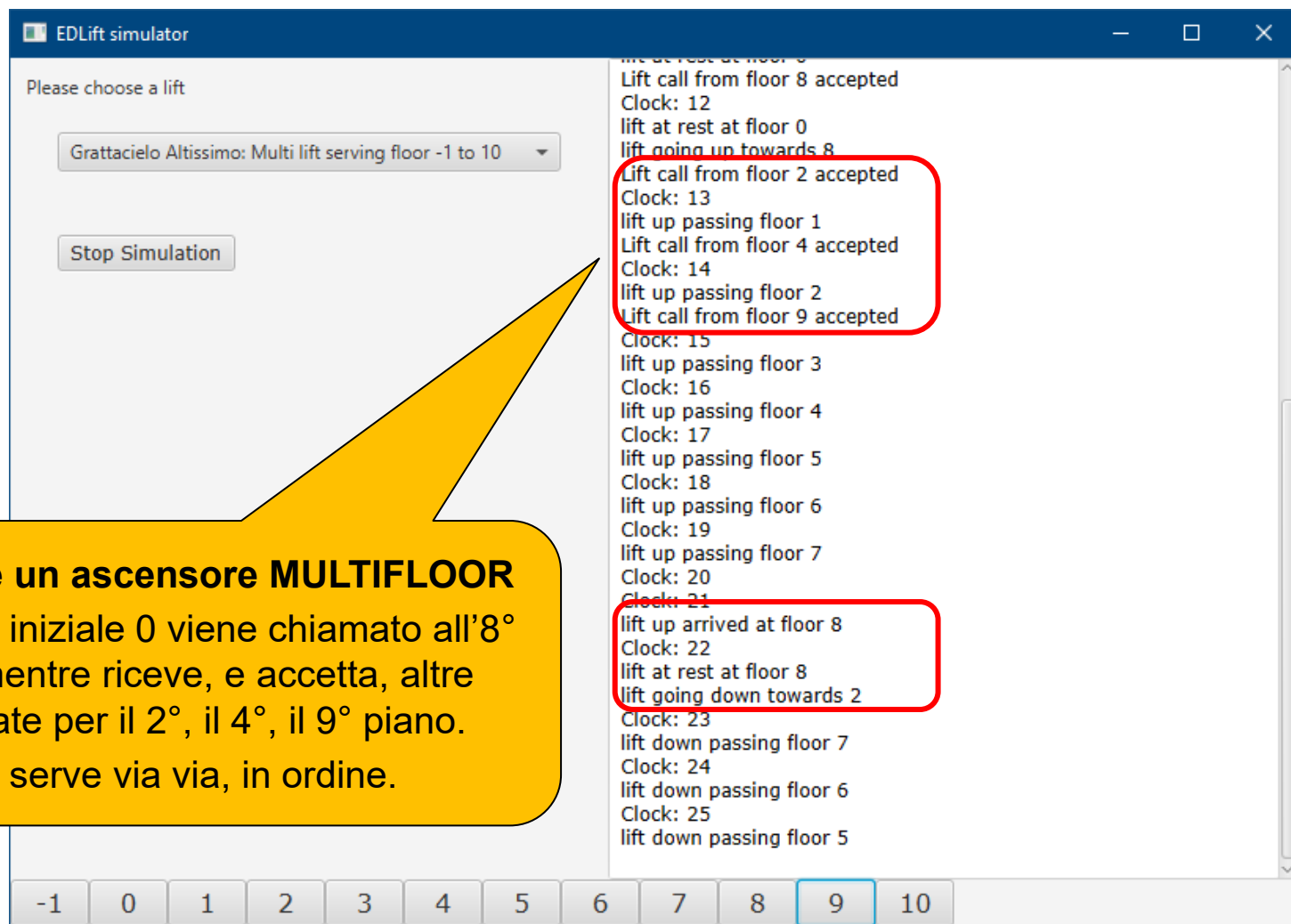
– SPECIFICA DI DETTAGLIO

- Il costruttore riceve la capacità massima (=dimensione array)
- Il metodo `insert` restituisce un boolean che indica l'esito dell'operazione (`true` = OK, `false` = inserimento fallito)
- Il metodo `extract` restituisce l'intero estratto
 - Se la coda è vuota, restituisce per convenzione `Integer.MIN_VALUE`
- Il metodo `size` restituisce il numero di elementi presenti in coda
- Il metodo `hasItems` restituisce vero se c'è almeno un elemento in coda
- Il metodo `peek` restituisce il prossimo elemento da estrarre (senza però estrarlo) *[non necessaria in questa esercitazione, ma utile per completezza di definizione dell'ADT]*

Architettura del model



Esempi di funzionamento



The screenshot shows the EDLift simulator window. On the left, there is a control panel with the text "Please choose a lift" and a dropdown menu currently set to "Grattacielo Altissimo: Multi lift serving floor -1 to 10". Below this is a "Stop Simulation" button. On the right, a log displays the lift's status and timestamps. A yellow callout bubble points to the log, containing text about a multifloor lift. Two sections of the log are highlighted with red boxes: the first shows calls from floors 2, 4, and 9 being accepted while the lift is at floor 8; the second shows the lift arriving at floor 8 and then going down towards floor 2. At the bottom, a row of buttons represents the floors from -1 to 10, with floor 9 currently selected.

EDLift simulator

Please choose a lift

Grattacielo Altissimo: Multi lift serving floor -1 to 10

Stop Simulation

lift at rest at floor 8
Lift call from floor 8 accepted
Clock: 12
lift at rest at floor 0
lift going up towards 8
Lift call from floor 2 accepted
Clock: 13
lift up passing floor 1
Lift call from floor 4 accepted
Clock: 14
lift up passing floor 2
Lift call from floor 9 accepted
Clock: 15
lift up passing floor 3
Clock: 16
lift up passing floor 4
Clock: 17
lift up passing floor 5
Clock: 18
lift up passing floor 6
Clock: 19
lift up passing floor 7
Clock: 20
Clock: 21
lift up arrived at floor 8
Clock: 22
lift at rest at floor 8
lift going down towards 2
Clock: 23
lift down passing floor 7
Clock: 24
lift down passing floor 6
Clock: 25
lift down passing floor 5

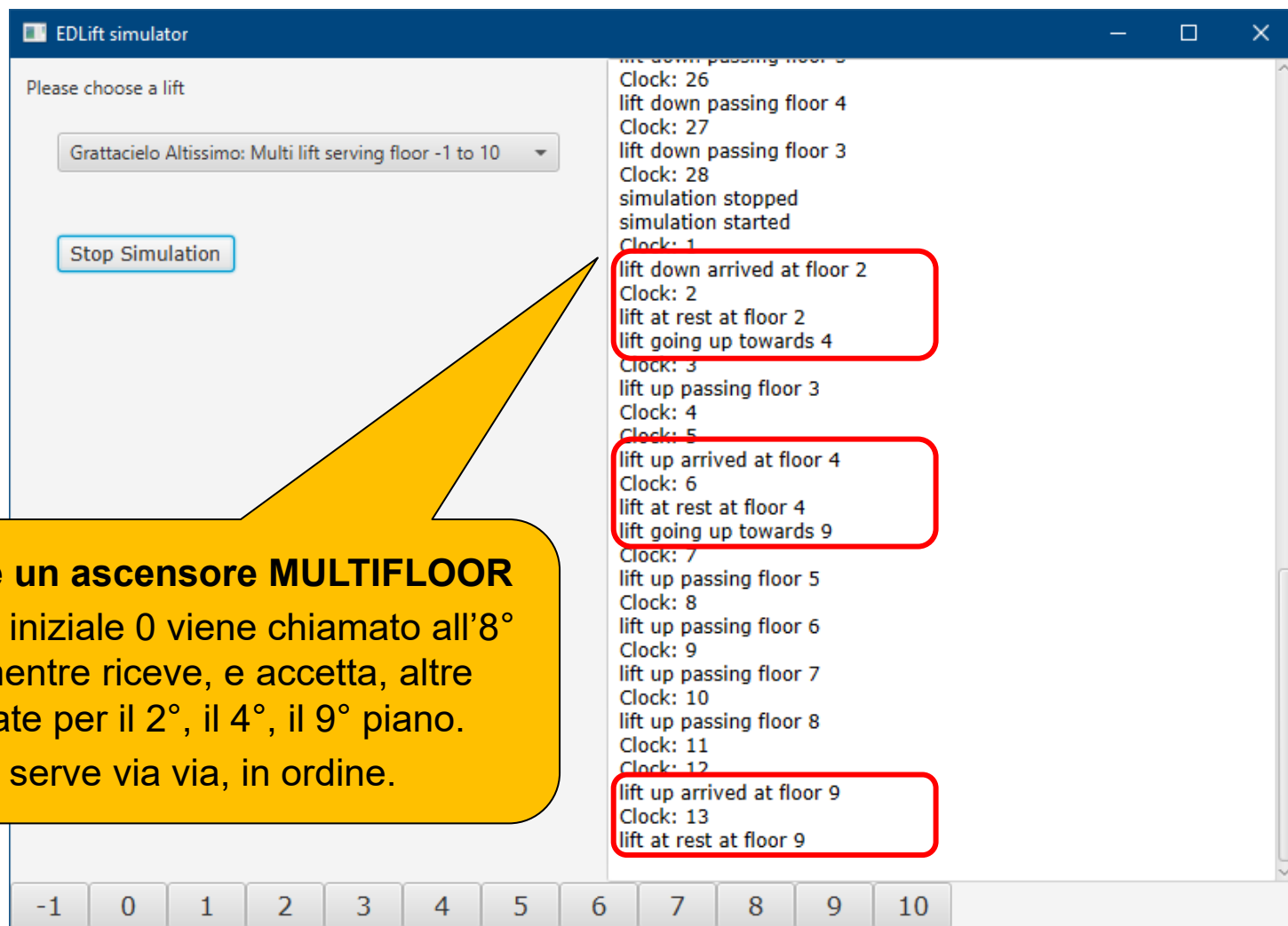
-1 0 1 2 3 4 5 6 7 8 9 10

Questo è un ascensore MULTIFLOOR

Dal piano iniziale 0 viene chiamato all'8°
e nel mentre riceve, e accetta, altre
chiamate per il 2°, il 4°, il 9° piano.

Le serve via via, in ordine.

Esempi di funzionamento



The screenshot shows the EDLift simulator window. On the left, there is a control panel with a dropdown menu set to 'Grattacielo Altissimo: Multi lift serving floor -1 to 10' and a 'Stop Simulation' button. On the right, a log displays the simulation's progress. A yellow callout bubble points to the log, explaining the multi-floor operation. Three specific log entries are highlighted with red boxes: the arrival at floor 2, the arrival at floor 4, and the arrival at floor 9. At the bottom, a row of buttons represents the floors from -1 to 10.

Please choose a lift

Grattacielo Altissimo: Multi lift serving floor -1 to 10

Stop Simulation

lift down passing floor 3
Clock: 26
lift down passing floor 4
Clock: 27
lift down passing floor 3
Clock: 28
simulation stopped
simulation started
Clock: 1
lift down arrived at floor 2
Clock: 2
lift at rest at floor 2
lift going up towards 4
Clock: 3
lift up passing floor 3
Clock: 4
Clock: 5
lift up arrived at floor 4
Clock: 6
lift at rest at floor 4
lift going up towards 9
Clock: 7
lift up passing floor 5
Clock: 8
lift up passing floor 6
Clock: 9
lift up passing floor 7
Clock: 10
lift up passing floor 8
Clock: 11
Clock: 12
lift up arrived at floor 9
Clock: 13
lift at rest at floor 9

-1 0 1 2 3 4 5 6 7 8 9 10

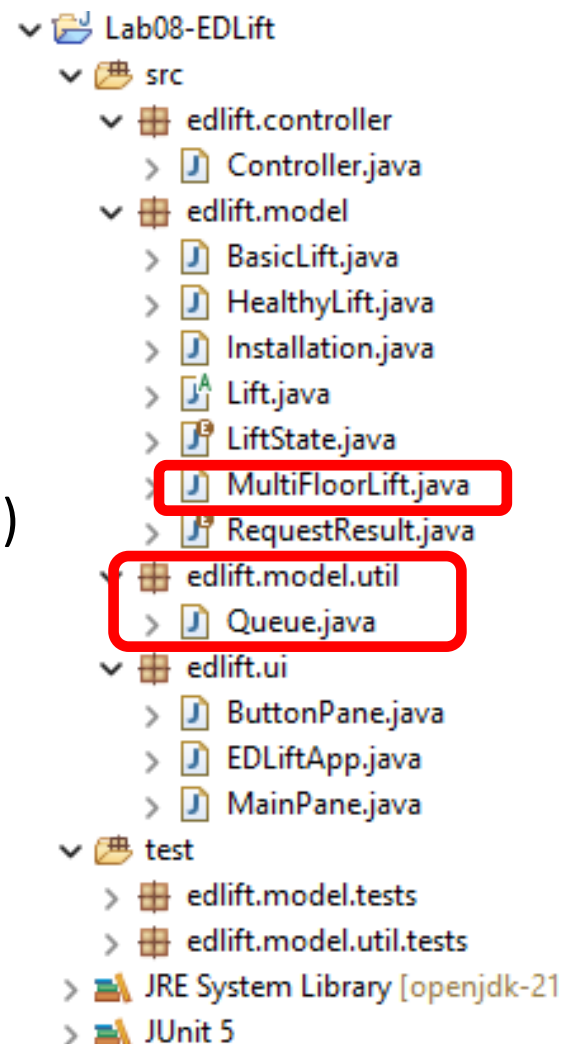
Questo è un ascensore MULTIFLOOR

Dal piano iniziale 0 viene chiamato all'8°
e nel mentre riceve, e accetta, altre
chiamate per il 2°, il 4°, il 9° piano.

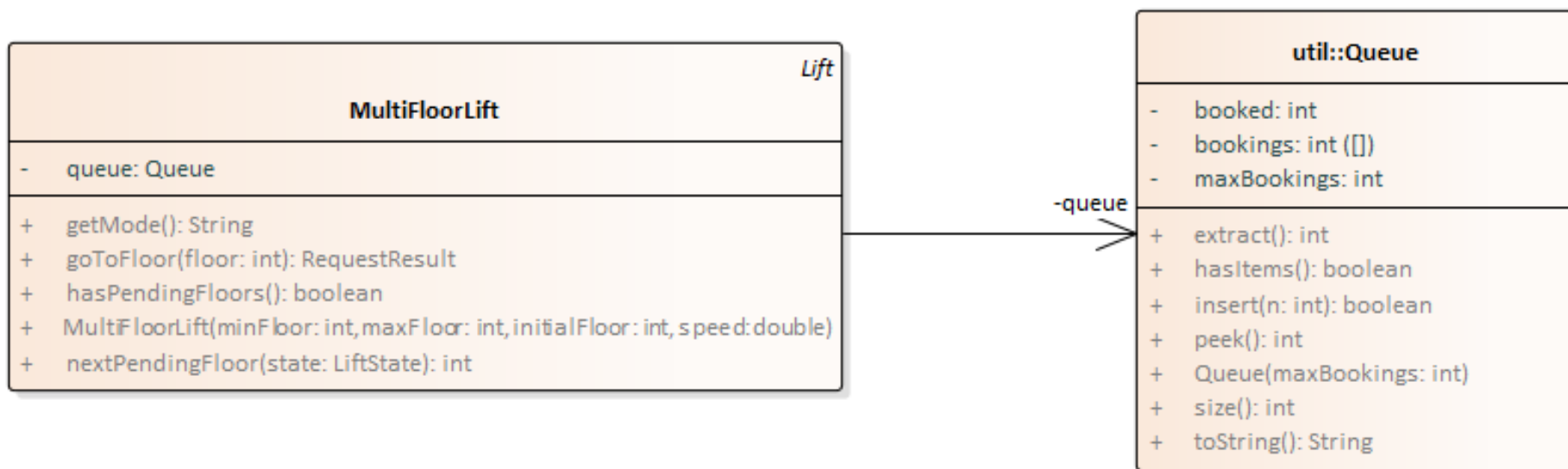
Le serve via via, in ordine.

Da fare (2ª parte)

- **Le classi da fare**
 - **MultiFloorLift**
 - **Queue**
- MultiFloorLift deve implementare:
 - **il metodo `getMode`**, che restituisce la stringa relativa al modo di funzionamento («MULTI»)
 - **il metodo-chiave `goToFloor`**
- **Test, test, test!**
 - Ci sono un sacco di test già pronti
 - Deve funzionare TUTTO!



Da Fare



Tempo a disposizione: tutto quello che rimane (e forse anche qualcosa in più 😊)

Hey!



**KEEP
CALM
AND
HAPPY
CODING**